

APIx / SAFAR

A trustworthy airfare index,
built on your own collection engine

ARCHITECTURE REVIEW + DETAILED DELIVERY PLAN / VERSION 3

Build a purpose-built Python collection framework inspired by Scrapling's separation of fetching, parsing, sessions and scheduling. Own the airfare contracts, adaptation logic, evidence trail and statistical pipeline. Do not import, wrap, vendor or fork Scrapling.

Recommended outcome A working multi-source prototype that turns permitted airfare observations into reproducible daily, weekly and monthly experimental indices, with every published value traceable to its evidence and methodology version.

Prepared from The supplied ps.md, the five-page APIx_SIH2026_Architecture_Plan_v2.pdf, and the current repository at commit 683bbcc, including uncommitted working-tree state. Scrapling was examined at revision b11da909c0544ea1c01a1a2e091166acf6e344e4 dated 29 September 2026.

Status of this document This is a proposed design and build plan, not a claim that the system or a live back-test has been completed. Thresholds, schedules and resource envelopes are proposed engineering defaults unless explicitly attributed to an external source. No live airline access, source permissions or official PSD weights were verified in this review.

The central change Make evidence quality, product comparability and reproducible publication the core of the project. A visually convincing chart is only the final output.

01 / Decisions and reading guide

What the team should build

Use a modular Python service with separately runnable workers, one PostgreSQL database, and an evidence directory that can later move to object storage. Keep the existing React/Vite interface. FastAPI serves versioned aggregate results. Begin with one permitted airline and one permitted OTA, then expand through identical adapter contracts to all sources named by the PS.

The team's original contribution is **SAFAR Collect**: typed airfare jobs, persistent sessions, drift detection, safe extraction recovery, quote provenance, and source health controls. Mature primitives such as Playwright, HTTPX, lxml and PostgreSQL remain dependencies; building your own framework does not mean building a browser, TLS library or HTML parser.

Priority decisions

Priority	Decision	Why it matters
P0	Separate LIVE, ARCHIVE and SYNTHETIC datasets	A simulation must never appear to be observed evidence.
P0	Obtain PSD routes/weights and verify the DGCA benchmark	These are explicit PS inputs, not optional decoration.
P0	Implement permissions, rate limits and evidence capture before live adapters	Both correctness and source access depend on them.
P0	Replace inferred tax splits and blanket quality adjustments	Unknown components and changed products cannot be made comparable by assumption.
P1	Introduce stable product slots and immutable index releases	Reruns must not rewrite already published history.
P1	Demonstrate selector drift with a local fixture	Proves resilience without requiring an uncontrolled live site change.
P2	Add forecasting and natural-language analysis only after validation	They cannot rescue an unverified source dataset.

How to use this document

Sections 02–06 establish requirements, gaps and sampling. Sections 07–14 specify the architecture, engine and data model. Sections 15–20 define the estimator, publication and validation. Sections 21–27 describe testing, operations, migration, implementation and the judging demonstration. Sections 28–29 contain primary-source references.

Definition of success: permitted daily extraction, auditable normalized records, PSD-configurable index calculation, a working dashboard/API, automated failure-case tests, and at least 30 days of authentic historical observations with a properly scoped independent comparison. Until the last item is available, describe the prototype as partially validated.

02 / Requirements mapped to evidence

The supplied PS is the acceptance contract. Its background claims are context, not independently verified measurements. Preserve its ambition while making each deliverable testable. [L1]

PS requirement	Planned implementation	Evidence needed to claim completion
Five airlines and leading OTAs	Adapter registry for IndiGo, Air India, Air India Express, Akasa Air, SpiceJet; MakeMyTrip, Yatra, EaseMyTrip, Cleartrip, Ixigo, Goibibo	Per-source permission record, fixture test and successful permitted live capture; report actual coverage.
Scheduled Python scraping	SAFAR Collect with HTTPX and Playwright workers	Scheduler log and persisted jobs from repeated daily runs.
JavaScript, CAPTCHA, anti-bot, IP/session handling	Browser transport; challenge classification; sticky sessions; approved egress policy	Tests for JS readiness, session expiry, 429, CAPTCHA and source suspension.
robots.txt / ToS / rate limits	Request policy gate and per-host quotas	Blocked-request audit, policy version and request timing trace.
Representative routes and all five horizons	Versioned basket with T+1, T+7, T+15, T+30, T+45	Configuration export, target matrix and weighted coverage report.
Clean database and fare components	Immutable raw records, normalized versions, component provenance	Trace from raw price to normalized price; duplicate and reconciliation tests.
Outliers, missing data, cancellation, sold-out	Separate observation states and documented quality decisions	Fixtures proving unavailable prices are never zero fares.
Index on PSD routes and weights	Basket import and frozen weight version	Supplied file, checksum, validation report and reproducible index run.
Daily, weekly and monthly APIx	One estimator; calendar roll-ups	Hand-calculated fixture and period-boundary tests.
Dashboard and consumer API	Existing React console + FastAPI read API	Daily index, sector heatmap, lead-time curve, OpenAPI and lineage walkthrough.
Documentation and automated testing	Adapter handbook, methods note, operational runbook and tests	Versioned documents and actual test output from the implemented system.
30 days against DGCA monthly average fares	Historical evidence manifest and independent benchmark import	Authentic dated observations, identified benchmark file, compatible scope and published comparison.

Interpretation: handling CAPTCHA means detecting it and following a documented recovery/access route, not guaranteeing a bypass. IP rotation is only an approved operational capability; it does not grant permission or justify resetting a blocked identity.

PSD dependency: the supplied files contain no authoritative PSD route/weight attachment. Load that attachment when supplied; use clearly named experimental weights until then.

03 / What the existing repository actually does

This review inspected source files, not a deployed service. Application tests were not run because the task is a planning deliverable; no production-readiness claim follows from this inspection.

Finding	Local evidence	Required change
Fare series are generated	<code>src/convex/lib/engine.ts</code> derives fares from date, route, carrier and epoch	Preserve the generator only for fixtures and an explicit demonstration mode.
“Run pipeline” simulates a collection	<code>src/convex/pipeline.ts:runSweep</code> invokes <code>simulateSweep</code> and increments epoch	Replace live-mode behavior with an authenticated job request and persisted status.
DGCA comparison is not independent	<code>src/convex/lib/index.ts:backtest</code> derives <code>dgcaIdx</code> and <code>dgcaFare</code> from its own monthly series	Remove this path from validation claims; import a real, independently sourced benchmark.
Historical baseline depends on epoch	<code>baseSnapshot(epoch)</code> recomputes generated prices	Store a frozen baseline and make later corrections explicit revisions.
Weights are indicative	<code>src/convex/lib/basket.ts</code> describes illustrative FY2024 traffic shares	Import an evidenced basket; passenger counts are not automatically expenditure shares.
Quality adjustment is assumed	<code>computeSeries</code> multiplies by a generated base/current quality ratio	Use stable products; defer fitted hedonic adjustment pending identifiable real training data.
Region and period logic need attention	Region accumulators are outside the daily loop; period year-on-year uses an offset of 12 for both weeks and months	Reset daily accumulators; use calendar-aligned comparisons and regression fixtures.
Daily “MoM” is misleading	Daily code compares adjacent days at a month boundary	Compute monthly change from completed monthly aggregates.
Python implementation is absent in this working tree	Git reports deleted <code>scraper/safar/*</code> ; remaining tests import those files	Treat rebuilding the collector as real work. Preserve the user's deletions until implementation decisions are made.

Assets worth retaining

The React pages already cover overview, sectors, lead time, explorer, pipeline, validation, methodology, API documentation and a presentation view. Reuse those page concepts and components. Retain the seeded generator for deterministic testing, but place synthetic data in a separate namespace with conspicuous labels and no live publication permissions.

Implication for the pitch: this is a good demonstration shell and a proposed system, not yet evidence of daily collection from eleven live sources or an independently validated statistical instrument. [L3]

04 / Changes to the v2 plan

v2 proposal	v3 decision	Reason
Postgres/Timescale, DuckDB, Parquet, multiple orchestration options	Postgres + evidence files; cron and a durable job table first	One authoritative operational store is easier to debug and deliver. Add analytic formats only when needed.
About 250 searches from 25 pairs × 2 directions × 5 windows	Correct to 250 per sweep; multiply by sweeps and eligible sources	Three sweeps mean 750 searches/source/day before retries and browser subrequests.
Cross-channel dedupe by flight/date/fare class	Dedupe repeated acquisition within source; link comparable offers across sources	OTA fees and bundles are genuine observations, not duplicate rows to delete.
Estimate tax components from inclusive totals	Preserve unknown components as null; keep observed inclusive total with fee-completeness status	Taxes and surcharges cannot be uniquely recovered from an unexplained total.
Subtract add-on baggage cost from included baggage	Use exact entitlement strata or add a purchasable upgrade to reach a specified product	The price of buying baggage is not the refund value of removing bundled baggage.
Airline direct is the source of truth	Report direct and OTA subindices; freeze a declared channel mix for the experimental combined series	Neither channel alone represents all consumer purchases.
Automatically remove MAD/IQR outliers	Flag unusual fares and exclude only when evidence supports an error	A real festival or capacity shock is precisely what the index must retain.
Jevons + Lowe/Laspeyres labels	Define matched product slots; label aggregation according to actual weights	A formula's name must match the data and estimator used.
Synthetic backfill calibrated to published statistics	Synthetic data for tests only; authentic archive or prospective collection for the PS	Simulated history cannot satisfy an observed-data back-test.
Method switcher, GEKS, nowcast, Ask APIx	Research layer after the core release is independently reproducible	Reduce scope while retaining the full required deliverable.
Compliance firewall stub late in the sprint	Working policy gate before the first live request	Source access decisions must control collection, not just appear on a slide.

Architecture options considered

A — Extend the current Convex simulation into production: fastest UI continuity, but browser execution and data lineage still need external services. Risk of keeping simulation and publication intertwined.

B — Python modular service + workers + Postgres: recommended. Collection, normalization and statistics share typed contracts while workers remain isolated from the API process. Reuse React; migrate data calls incrementally.

C — Kafka, distributed schedulers, many microservices and a warehouse: appropriate only after measured scale demands it. More deployment and failure modes than this pilot needs.

Source of reviewed proposals: supplied v2 PDF, sections 2–9. [L2]

05 / Resolve external data before polishing charts

Current CPI context

MoSPI's first CPI release on base **2024=100**, dated 12 February 2026, states that weights use HCES 2023–24 and the classification uses 12 divisions. It also describes expanded digital/administrative price inputs, including airfare. Therefore, do not repeat the PS's older “Transport and Communication” framing as an independently verified description of the current CPI. Preserve the PS wording and separately document the applicable current classification. [S7]

Keep APIx's pilot reference period distinct from the official CPI base year. Do not label pilot observations “2024=100” without genuine 2024 prices or a documented linking method. A rebased chart does not make different statistical scopes identical.

Three separate data needs

Input	Purpose	Status and next action
PSD basket and weights	Acceptance configuration	Not present in supplied materials. Obtain organizer/PSD file; record units, directionality, reference period and checksum.
DGCA origin–destination passenger traffic	Route selection and an explicitly named traffic-weighted sensitivity series	Exact dataset not verified in this review. Confirm whether counts represent passenger segments or journeys, directional or combined flows. Airport totals alone cannot establish city-pair flows.
DGCA monthly average-fare benchmark	Independent validation required by PS	A suitable downloadable table was not verified. DGCA portal access failed in this research environment; absence of verification is not proof of absence.

Day-one data gate

Assign a data owner to produce data_sources/register.csv: publisher, title, URL, downloaded filename, SHA-256, publication date, covered dates, unit, geography, cabin, taxes, aggregation and access conditions. Preserve each original file and parser version. Double-check a sample of extracted cells against the publication.

If the required DGCA fare table cannot be obtained, seek clarification from the organizer/PSD and keep that acceptance item **unmet**. MoSPI airfare item indices may provide supplementary external checks, but they do not automatically replace the requested DGCA comparison. eSankhyiki is the PS's nominated discovery portal, not proof that any particular airfare table exists. [S8, S12]

Claims to verify before the final deck: online ticket share, within-day fare ranges, representative traffic coverage, all source permissions, and any claimed CPI contribution. Attribute PS background statements to the PS if no independent evidence has been obtained.

06 / Sampling frame and comparable product

Freeze what is being priced

Initial product definition: one adult; one-way domestic economy; nonstop; INR; fixed Indian point of sale; logged-out, non-personalized session; general-public fare; no loyalty, student, corporate or bank-card eligibility. Record exact refundability and baggage entitlements. Do not assume every carrier sells an identical product.

Use approved product strata, for example 7 kg cabin + 15 kg checked baggage with a specified refundability category. An adapter must observe those entitlements or observe an available upgrade that reaches the target. Higher bundled entitlements remain a separate stratum unless PSD approves their treatment.

Exclude product combinations that cannot be made comparable; report their weight as uncovered.

Collection and index units

Unit	Definition
Route	Directed airport pair, e.g. DEL→BOM; direction is never inferred from a symmetric display label.
Search job	Source × route × departure date × lead days × product × observation slot × basket version.
Quote identity	Source offer + itinerary + departure timestamp + fare family/product + observation job.
Statistical slot	Route × lead window × carrier × departure-time band × product stratum × source/channel allocation.
Release	Dataset kind + observation date/period + basket, method, baseline and revision versions.

Search T+1, T+7, T+15, T+30 and T+45 using the **Asia/Kolkata observation date**, not elapsed UTC hours. Revalidate lead days at acquisition; a job that crosses midnight must be expired/replanned, not mislabeled. Departure-date cohorts move daily: this is a booking-date offer index, not a realized ticket-sales index.

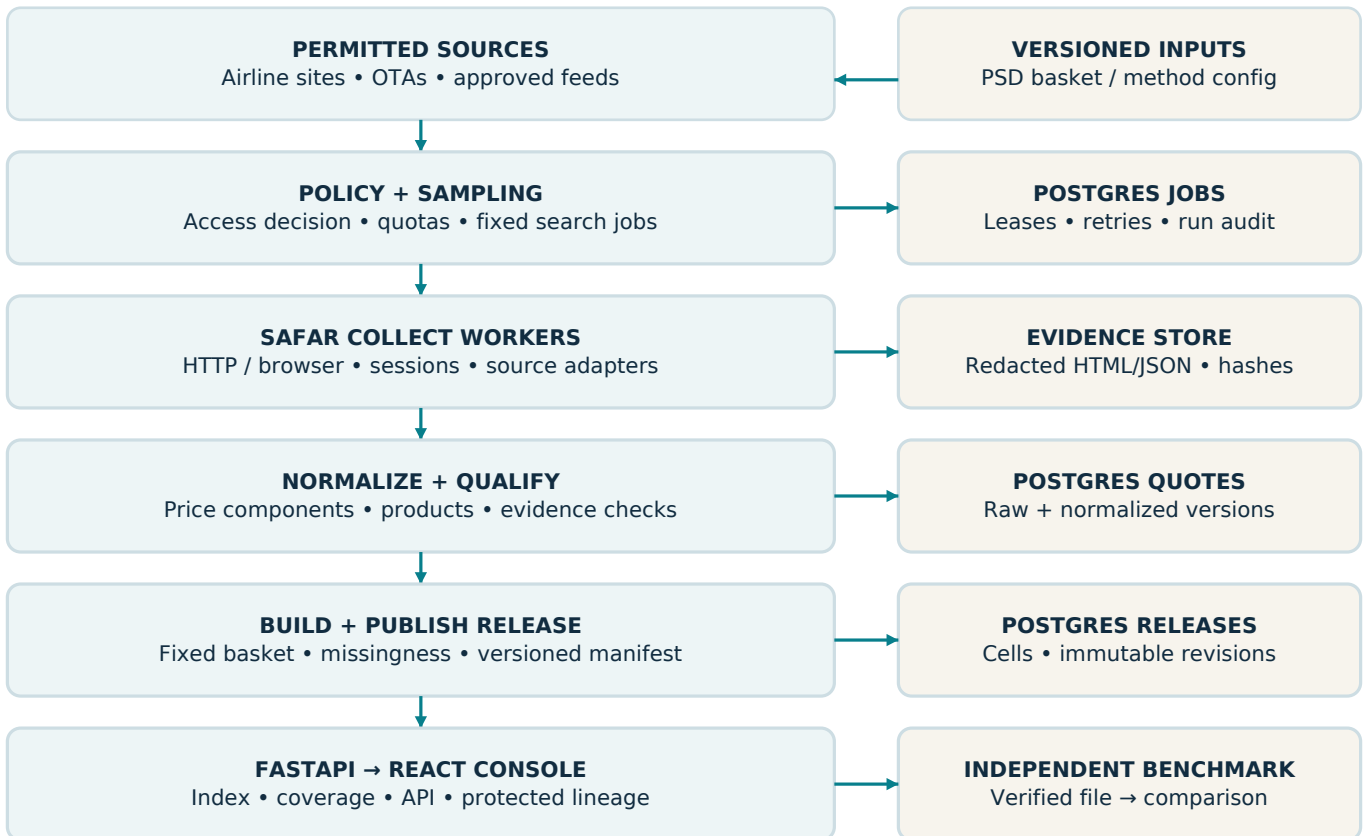
Pilot and expansion

Start with the six example directions named by the PS: DEL-BOM, DEL-BLR, BOM-BLR, DEL-CCU, BLR-HYD and MAA-DEL, plus their reverse directions: 12 directed routes. This is an engineering pilot, not a claim that these are the official basket. PSD routes take precedence when available.

Start at one fixed daily observation window, 10:00–12:00 IST. Expand to 07:00–09:00, 13:00–15:00 and 19:00–21:00 only when permissions and capacity allow; changing the design creates a new sampling version. Within each window rotate route order using a reproducible seed and small jitter. Keep departure-time bands separate from collection windows.

Freeze target slots using observed baseline support. For provisional departure bands use 00:00–08:00, 08:00–16:00 and 16:00–24:00 IST. Store a capability matrix so non-operated routes are not repeatedly searched without purpose. A source outage changes achieved coverage, never the original denominator.

07 / Recommended system architecture



All Postgres boxes refer to one authoritative database; evidence bytes live separately.

Boundaries and ownership

Control plane: basket configuration, source permissions, scheduling, job leases, source health and publication approvals. **Data plane:** fetch, parse, preserve evidence, normalize, qualify and aggregate. The API never launches an interactive browser inside a web request.

PostgreSQL is authoritative for jobs, normalized data, policy decisions and release metadata. Large HTML/JSON snapshots live in content-addressed evidence storage. A local encrypted volume is sufficient for the pilot; move behind the same storage interface to object storage later. Files are not placed in public frontend assets.

The release builder reads a fixed input manifest and writes candidate cells. It then atomically publishes a release record referencing those cells. The API and dashboard read published release IDs, not a fresh estimator invocation on every request. Raw evidence access is restricted separately from public aggregate data.

External inputs: PSD configuration and independent benchmark files enter through validated importers. They never flow through the synthetic generator. A separate SYNTHETIC workspace uses the same contracts for demonstrations, but is excluded from LIVE and ARCHIVE manifests by database validation.

Scale choice: one codebase, separate worker/API processes. Do not introduce Redis, Kafka, Timescale or Prefect in the pilot unless a measured requirement justifies the extra service. Read-only DuckDB/Parquet analysis may later consume exports without becoming a second source of truth.

08 / SAFAR Collect: own the framework

What Scrapling informs

Scrapling documents separate HTTP/browser fetchers, session support, adaptive selection and crawler controls. Its parser includes relocation logic; its storage module persists element descriptors; its scheduler tracks queued/in-flight work and checkpoints; its session manager manages named sessions. These are useful architectural references, not airfare-validation guarantees. [S1–S5]

Reference concept	Your implementation	Airfare-specific addition
Fetcher abstraction	HttpTransport and BrowserTransport	Every fetch carries query context, policy verdict and acquisition time.
Persistent sessions	SessionPool keyed by source, locale, product context and egress	Prevent source identity or point-of-sale changes within an observation.
Adaptive parser	SelectorRegistry + constrained candidate matching	Flight, date, currency, product and price semantics are hard gates.
Descriptor storage	Versioned fingerprints and approved adapter versions	Never overwrite a working selector automatically from a failed page.
Queue / checkpoints	PostgreSQL jobs, leases and idempotency keys	Search body/date/horizon participate in identity; URL alone is insufficient.
Replay development	Immutable evidence fixtures and replay transport	Old evidence keeps its original observation time.

Build versus reuse

Build: job and quote models, adapter interface, source state machine, domain policy, selector scoring, eligibility rules, evidence manifests, release logic and operational metrics.

Reuse as ordinary dependencies: Python asyncio, HTTPX, Playwright Chromium, lxml/cssselect, Pydantic, SQLAlchemy/Alembic, PostgreSQL, FastAPI and pytest. Pin exact versions and browser revisions during implementation; commit a lock file after compatibility testing. Do not add Scrapling to runtime or development requirements.

Do not reimplement a generic web crawler, TLS fingerprint impersonation suite, CAPTCHA solver, proxy marketplace, MCP server or universal scraping DSL. They are outside this PS's core engineering requirement. Your differentiator is a collection system that refuses to turn an uncertain extraction into a confident price observation.

Originality record: document which ideas were studied and the independent implementation decisions. No upstream code is copied in this plan. Scrapling's repository carries a BSD 3-Clause license; if the team later elects to reuse code, preserve applicable notices and describe that reuse accurately. [S6]

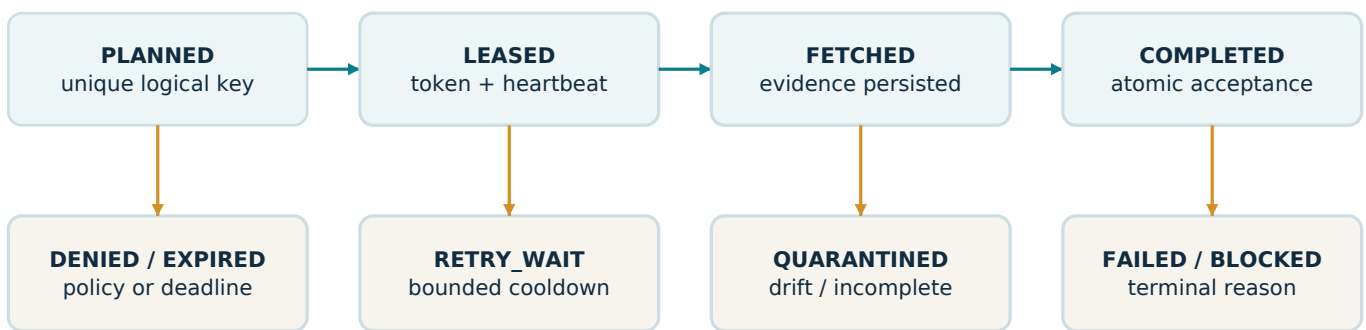
09 / Durable jobs and engine contracts

Proposed interfaces

```
Planner.plan(basket, observation_window) -> list[SearchJob]
PolicyGate.evaluate(job, request_url) -> PolicyDecision
Transport.fetch(job, session) -> FetchResult
SourceAdapter.extract(fetch_result, job) -> ExtractionBatch
Normalizer.normalize(raw_quote, method) -> NormalizedQuote
QualityGate.evaluate(quote, history) -> QualityDecision
ReleaseBuilder.build(manifest) -> CandidateRelease
Publisher.publish(candidate_id, reviewer) -> Release
```

SearchJob contains job ID, idempotency key, source, route, departure date, lead days, passenger/product specification, observation window, attempt, deadline, basket version and policy version. FetchResult includes final URL, status, captured-at time, network timing, evidence IDs, content hash and failure classification. ExtractionBatch includes offers, availability state, selector version and field-level provenance.

Lifecycle



Failure classifications depend on stage; only retryable states return to the queue within deadline.

The database enforces a unique logical-job key. Workers claim jobs using a short transaction and row locking; SKIP LOCKED can support competing consumers. Commit the claim before network work, then renew a lease with a heartbeat. A stale worker must present the current lease token before completing. [S11] Use a 180-second initial lease, 30-second heartbeat and 120-second attempt budget as pilot defaults. A worker crash makes the job claimable after lease expiry. Commit raw quote rows, evidence references and terminal state together; write evidence first by content hash, then commit metadata. Unreferenced objects can be reclaimed later.

Retry policy: transient timeout/5xx gets at most two retries, with approximately 30 and 120 seconds of backoff plus jitter, bounded by the job deadline. Honor a longer Retry-After. A 429 cools the host queue. A CAPTCHA/access block suspends automatic attempts for that source pending the source policy; no immediate identity change and retry. Schema drift goes to extraction quarantine.

Exactly-once claim: delivery is at least once. Uniqueness constraints make accepted observations and release inputs idempotent; the design does not claim exactly-once network execution. Late results remain evidence but do not enter a closed observation window. Replays never become new live observations.

10 / Fetching, sessions and source access

Onboard every source through the same gate

Create a registry entry with owner, allowed hosts/paths, permitted collection method, ToS review record, permission/contract evidence where required, robots snapshot/hash, expiry, rate budget, allowed observation hours and kill-switch status. A candidate source starts UNVERIFIED, not ALLOWED. robots.txt controls crawler behavior; it is not access authorization. RFC 9309 explicitly separates those concerns. [S9] Before the first request, resolve policy. Repeat checks for redirects and newly encountered hosts. Treat inaccessible or expired policy evidence as a pause under this project's conservative default. Recheck robots at least daily and after a policy-change signal. Preserve the policy decision with each attempt. Do not claim this is a legal determination of source terms.

Transport decision

1. Prefer a permitted documented feed/API when its product scope matches the observation; label the acquisition method. It supplements the required web-scraping demonstration rather than silently replacing it. 2. Use HTTPX where a permitted response already contains sufficient structured data or HTML. 3. Use Playwright for JavaScript search flows. Wait for an adapter-specific results-ready condition, verify echoed route/date/passenger inputs, and cap page/network waits. 4. Capture relevant browser response bodies only within the allowed workflow and host policy. Playwright provides response events and request interception; access permission remains a separate decision. [S10] 5. Extract structured data first, then approved DOM selectors. Never invent an undocumented endpoint to bypass a refused workflow.

Session and egress rules

Maintain isolated browser contexts per source and collection identity. Set locale, timezone, currency and point of sale explicitly. Keep cookies in memory or encrypted storage with a short lifetime; redact tokens from logs and evidence. A context expires on logout, invalid session, policy change or end of its allowed lifetime. Do not store traveler names or payment details.

Use a declared stable egress identity by default. Support an approved egress pool only for permitted reliability/load distribution, with sticky identity for the complete search. Quotas apply across the entire pool. Never switch IP simply to defeat a source's block or reset its rate limit.

Set an initial ceiling of one active search per source and one new search every 30 seconds, subject to any stricter source limit. Separately meter browser subrequests and relevant API calls: a search is not one HTTP request. Observe actual request volume before increasing concurrency. A global and per-source kill switch must prevent new requests immediately and cancel safe in-flight work.

11 / Adaptive extraction without silent errors

Detect change even when a selector still matches

Check required-field completeness, result cardinality, query echo, expected currency, itinerary/date consistency, fare label context and component reconciliation on every run. A selector returning a number is not proof it returned the fare. A “from” advertisement, struck-through price, monthly installment or fare for another passenger count must fail the semantic gate.

Store fingerprints per source, page type, locale, product and field: stable attributes, accessible labels, DOM ancestor structure, sibling labels and relative position inside the matched itinerary card. Drop volatile generated classes and session IDs. Keep price magnitude out of the similarity score so a legitimate price jump does not look like a parser failure.

Proposed recovery algorithm

1. Parse with the approved selector/schema version.
2. Validate query identity, card identity and field semantics.
3. If invalid, classify drift and preserve the original evidence.
4. Search candidate fields only inside the correct itinerary card.
5. Score: .30 label + .25 stable attributes + .20 ancestor shape + .15 sibling context + .10 relative location.
6. Require score $\geq .90$ and lead over runner-up $\geq .15$, plus all semantic checks. Otherwise abstain.
7. Write a candidate extraction and selector proposal to quarantine.
8. Reviewer accepts only after old/new fixtures and a held-out test pass.
9. Publish adapter version; replay evidence with original timestamps.

These weights and thresholds are proposed starting values, not Scrapling parameters or measured accuracy. Calibrate them on labeled fixtures. If signals are absent, assign zero rather than renormalizing a weak candidate to high confidence. The score is a ranking heuristic, not a probability.

Demonstration and evaluation

Build at least 100 labeled page variants across both pilot source adapters: CSS renaming, wrapper insertion, card reorder, missing taxes, swapped currencies, decoy prices, stale query echo, multiple fare families and challenge pages. Split by original page family to prevent near-duplicate leakage.

Measure precision among accepted recovery candidates, eligible-change recovery rate, abstention rate and false acceptance of hard negatives. Proposed promotion gate: at least 99% candidate precision on the held-out labeled set and zero wrong-itinerary/wrong-currency acceptances; report denominators and limited sample size. An ambiguous pair must abstain regardless of score.

For the first pilot, no new selector auto-publishes live prices. After review, replay can create a corrected extraction version and an explicit release revision. Scrapling's parser and descriptor store inspire the separation; this constrained workflow is your own proposed design. [S2, S3]

12 / Normalize price meaning, not just text

Canonical money and components

Store amounts as integer paise (or a fixed decimal type), with currency and per-passenger/per-party basis. Keep the displayed text as evidence. Empty, dash, “sold out” and parse failure become null values with reasons; never zero. Reject negative payable fares and impossible currency/basis combinations from the index.

```
consumer_payable = base + carrier_surcharge + airport_charges
                  + government_taxes + mandatory_channel_fees
                  + observed_required_product_upgrades
                  - unconditional_public_discount
```

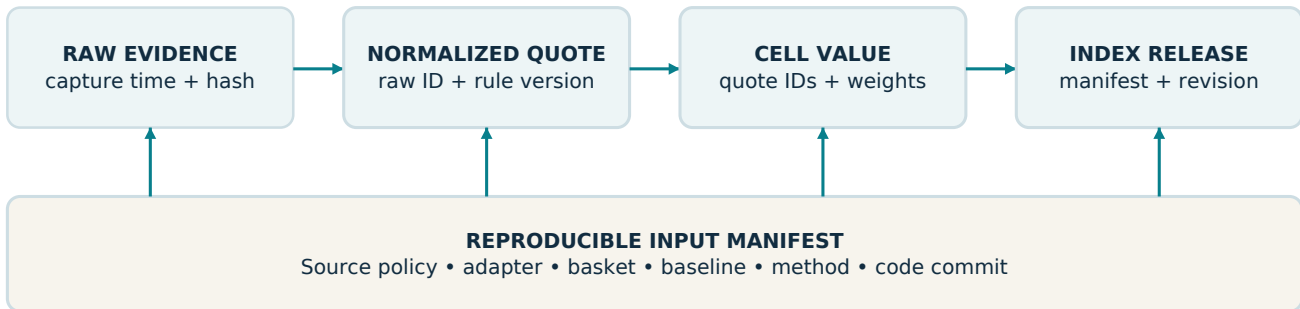
Components may be source-specific subitems to prevent double counting a surcharge already included in base or tax. Each component has OBSERVED, UNKNOWN, or NOT_APPLICABLE provenance and an evidence location. A derived sum is labeled DERIVED_FROM_OBSERVED; it is not a separately observed displayed total. When an inclusive total is visible but the breakdown is absent, preserve the observed total and leave base/taxes unknown. It may enter the payable-price series **only if** mandatory-fee and product completeness are established. It cannot enter a tax-exclusive series. Never back out GST or airport fees from a generic percentage table to fabricate observed components.

What the consumer can actually purchase

Situation	Treatment
Base and all mandatory charges shown	Reconcile sum to final displayed amount within INR 1 rounding tolerance.
Search price excludes unknown checkout fee	HEADLINE_ONLY; keep for diagnostics, exclude from payable headline until resolved.
Mandatory fee depends on payment method	Use a fixed documented broadly available payment scenario, without entering personal/payment data. If unavailable, mark incomplete.
Bank, loyalty or user-specific coupon	Store eligibility metadata; exclude discount from the general-public product.
Automatically available public discount	Include when its conditions match the fixed product and session.
Missing baggage entitlement	Ineligible for that product stratum until confirmed.
Checked bag can be purchased to reach the reference tier	Add only the actually observed purchasable fee for the same itinerary.
Higher baggage tier bundled into price	Keep a distinct product stratum; do not subtract a hypothetical baggage value.

Worked example: base INR 4,000 + carrier surcharge 300 + airport charges 500 + taxes 250 + mandatory channel fee 150 – unconditional discount 100 = INR 5,100 payable. If only INR 5,100 and complete-product evidence are visible, store total 510000 paise and null component fields. Confidence scoring cannot make missing components known.

13 / Data model and provenance



Entity	Required content / constraint
source_policy	Source, version, hosts, permissions/ToS evidence, robots hash, expiry, rate budget, status.
basket_version / basket_slot	Routes, products, horizons, schedules, slot weights, source references, effective dates, baseline link; weights sum to one.
collection_job / attempt	Unique logical key; lease token; target date/window; attempt timings; final state; policy version.
raw_evidence	SHA-256, content type, redacted URI, captured_at, source, rights/retention class, byte count.
raw_quote	Immutable observed fields/text, itinerary and offer IDs, job/attempt IDs, dataset_kind, evidence ID.
normalized_quote	Raw ID, normalization version, canonical values, component provenance, product ID, eligibility and rejection reasons. Unique raw ID + version.
selector_version	Source/page/field fingerprints, parent version, approval record, fixture report.
cell_value	Date, slot, eligible quote IDs, aggregation method, price, observed/imputed status, donor lineage.
index_release	Period, revision, status, input manifest, code/config hashes, baseline/basket/method versions, coverage, previous release ID.
benchmark_import	Independent original-file hash, publisher, dates, scope, extracted values, importer version.

Non-negotiable fields on each normalized quote

Origin, destination, operating and marketing carrier when available, flight number/segments, departure timestamp and timezone, observed_at UTC, observation_date IST, derived lead days, fare family, cabin, refundability, cabin/checked baggage, passenger count, channel, source, price basis, base fare, component list, displayed total, payable total, fee completeness, availability and evidence ID.

Distinguish flight instance from statistical slot: departure date belongs to the former; fixed lead days and product specification belong to the latter. A slot must persist across observation dates while the actual departure date moves.

Storage rules: append raw observations; normalization changes create a new version. Restrict raw evidence access. A reviewer decision records who, when, why and before/after values. Partition large quote tables by observation month only after measured size warrants it. Index job status/deadline and quote source/route/date/lead. Daily replay should require the manifest and retained evidence, not a fresh network request.

14 / Cleaning and quality decisions

Deduplication preserves channel economics

Drop exact ingestion duplicates using source offer identity, full itinerary/product, observation job and content fingerprint. Do not dedupe a fare observed tomorrow against today's fare. Keep different OTA prices for the same flight; link them under a comparable-offer group with baggage, refundability, payment assumptions and timestamps. Deduplicate code-share listings only when they are genuinely the same purchasable offer. A cross-source spread should use observations within a proposed 10-minute tolerance and the same itinerary/product. Outside that tolerance, mark the spread asynchronous and avoid attributing it solely to fees or discounts. One OTA syndicating another's inventory is not an independent statistical confirmation.

Separate availability from extraction failure

State	Meaning and index treatment
AVAILABLE	Eligible observed fare can contribute.
SOLD_OUT	Explicit unavailable inventory for the product; no price, preserve availability signal.
CANCELLED	Explicit source evidence of cancellation; never infer cancellation from absence.
NO_RESULTS	Valid search returned none; no claim about why.
NOT_OPERATED	Documented route/service absence; tracked in basket maintenance.
BLOCKED / TIMEOUT	Collection failed; unknown market price.
PARSE_DRIFT / INCOMPLETE_PRICE	Evidence acquired but insufficient for a qualified fare.

Outliers and confidence

Use log-price MAD alerts within comparable route/lead/product/channel groups only when there are at least 10 valid recent observations. Otherwise apply semantic checks and review without claiming a stable outlier distribution. A suggested alert is more than 5 robust deviations; it triggers inspection, not automatic deletion. Preserve events that are supported by the evidence.

Track separate dimensions: query correctness, product completeness, fee completeness, extraction method, freshness and reconciliation. A numerical summary may help triage, but eligibility uses hard rules and each rule has a visible reason. Never dynamically downweight expensive quotes because they disagree with other sites.

Error versus event example: a stale DEL-BOM search returning BOM-DEL results is a hard failure. A valid DEL-BOM fare rising from INR 6,000 to INR 12,000 with matching query/product and complete payable total remains an observation, even if unusual.

Reconcile counts at every boundary: planned jobs → terminal attempts → raw offers → unique offers → eligible offers → observed slots → published weighted coverage. "Scrape success" alone is not a data-quality metric.

15 / A precisely defined experimental estimator

Construct stable prices first

For each fixed statistical slot and observation window, take the median eligible payable fare across its unique flight offers. Require the product, carrier, channel/source allocation and departure band to match the slot definition. Record the number of flights and availability fraction. Median fares are a proposed robust offer-price estimand; they are not passenger-weighted transaction averages.

If there are multiple daily observation windows, take their geometric mean using fixed equal window weights. A missing window counts as missing/imputed under section 16; do not average only the convenient successful windows. Fix one-window or three-window design in the sampling version.

Freeze a baseline after the first **seven complete consecutive pilot observation dates** that pass coverage gates. Geometric-mean each slot's prices over these dates. Publish preliminary baseline status until frozen, then compute releases consistently against it. Prefer a longer reference period for deployment after PSD review. Do not reconstruct the baseline when a new collection epoch runs.

Elementary and upper aggregation

```
p(s,d) = qualified daily price for fixed slot s on date d
p(s,0) = frozen geometric-mean baseline for slot s
r(s,d) = p(s,d) / p(s,0)

J(c,d) = exp( sum over s in cell c [ a(s|c) * ln r(s,d) ] )
        with fixed a(s|c) >= 0 and sum a(s|c) = 1
```

```
APIx(d) = 100 * sum over cells c [ W(c) * J(c,d) ]
        with fixed W(c) >= 0 and sum W(c) = 1
```

Cell c is route × lead window × product × channel. Internal slots contain carrier, departure band and source allocation. Equal internal weights give a Jevons-type elementary index on matched slot prices; unequal fixed weights give a weighted geometric variant. The upper level is a fixed-weight arithmetic aggregation of cell relatives. Call it **experimental fixed-weight APIx** unless the input weights and reference periods justify a more specific CPI estimator name. [S13]

Worked check: cell A has two slot relatives 1.10 and 0.90, so its equal-weight geometric relative is 0.994987. Cell B is 1.20. With cell weights 0.60 and 0.40, $APIx = 100 \times (0.60 \times 0.994987 + 0.40 \times 1.20) = \mathbf{107.6992}$. At baseline, all relatives equal 1 and APIx is 100.

No hand-assigned hedonic quality multiplier. Fixed products address major comparability differences; residual changing flight mix is a disclosed limitation and a later research question.

16 / Weights, missing prices and coverage

Weight hierarchy

Import PSD weights as supplied, including their scope and reference period. If they cover only routes, additional lead/product/channel allocations must be declared separately; do not present those additions as PSD instructions. Expenditure shares and passenger shares answer different questions. Passenger traffic can support a traffic-weighted experimental series, but does not become household spending merely by being normalized. [S13]

Until official allocations exist, a fully declared pilot may use equal route weights, 0.20 per lead window, one reference product, and 0.50 direct / 0.50 OTA channel shares. Split each channel across approved sources and supported carrier/band slots using fixed baseline allocations. These are **sensitivity assumptions**, not estimated consumer purchasing shares. Freeze them before comparing dates. Also publish direct-only and OTA-only results so the channel assumption is visible.

If expenditure proxies are explored, calculate passenger count $\times$ independently evidenced reference-period average price and label the result an estimated expenditure weight. Never substitute current scraped quote counts as weights.

Missing-price hierarchy

1. Use another observed quote only if it belongs to the same fixed slot; never switch channel/product silently. 2. If a slot was observed yesterday, impute its relative movement from at least three observed donor slots spanning three routes, matched on lead, product and channel within a predeclared route group. Use the donors' geometric mean day-over-day relative. 3. If valid donors are unavailable, permit unchanged carry-forward for one calendar day, flagged prominently. No more than two consecutive days of any imputation for a slot. If no defensible value exists, the required release is suppressed. 4. Permanently missing products require a reviewed basket revision and overlap/linking study. Sold-out fares remain unavailable observations; any imputation is a statistical assumption, not a reconstructed offer.

Publication gate: proposed pilot defaults

Define slot weight $\omega(s) = W(c) \times a(s|c)$. Observed weighted coverage is the sum of weights with fresh qualified observations; imputed weight is reported separately. Do not silently renormalize weights after outages.

Publish a daily release only when all fixed weight is represented by observed or permitted imputed values, observed weighted coverage is at least 80%, imputed weight is at most 20%, every route has at least 60% observed conditional weight, and T+1 and T+7 each have at least 70% observed conditional weight.

Otherwise show SUPPRESSED_INSUFFICIENT_COVERAGE and the reasons. No synthetic data fills gaps.

These gates require empirical tuning and PSD agreement before official use. Report sensitivity to alternative weights, missing-data rules and channel choices alongside the main experimental series.

17 / Periods, revisions and statistical interpretation

Daily, weekly and monthly outputs

The daily index belongs to the IST date on which a consumer queried future departures. Publish after that date's configured observation windows close; proposed final pilot cutoff is 22:00 IST, with initial release by 23:00. The live collection status may update sooner, but a completed daily index is not a continuously observed market price.

Weekly output is the arithmetic mean of daily indices over an ISO Monday–Sunday week. Monthly output is the arithmetic mean over the calendar month. Use unsmoothed daily releases. Display a trailing seven-day moving average as an additional visualization, never substitute it silently for the daily input to monthly statistics.

A complete period requires every constituent daily release to pass its gate. During collection, label period-to-date values PROVISIONAL with day coverage; do not compare an incomplete current month to a complete prior month as a finalized MoM statistic. If days remain suppressed, the complete-period result is suppressed pending a documented revision policy.

```
MoM(m) = 100 * (MonthlyIndex(m) / MonthlyIndex(m-1) - 1)
YoY(m) = 100 * (MonthlyIndex(m) / MonthlyIndex(m-12) - 1)
Cell contribution to index-point move = 100 * W(c) * [J(c,t)-J(c,u)]
```

Missing comparable periods produce null, not zero. Calendar lookups must handle leap days and ISO week-year boundaries; do not use a 12-observation offset for weekly YoY. Suppress weekly YoY until an explicit calendar-alignment convention is implemented and documented.

Immutable releases

Each publication includes release ID, revision, as-of time, input manifest hash, basket/weight/product/method versions, code commit, baseline period, observed and imputed weights, suppression reasons and previous revision ID. A retry cannot alter a published revision. Corrected extraction creates a candidate revision with a human-readable reason and before/after impact.

No automatic chain linking in the pilot. A future basket refresh needs an overlap period and a versioned link factor approved by the methodology owner. Keep unrevised vintages downloadable.

Interpret the charts correctly

Lead-time ratios across T+1...T+45 are descriptive booking-window curves. Their log slopes are not causal demand elasticity. A panel following the same departure cohort across days is a separate analysis. Fuel, festivals and disruptions can be annotated with sourced dates; a coincident price increase does not establish causation. Nowcasting needs a genuine benchmark and a longer independent history, so it is deferred.

18 / Read API and access design

Proposed public contract

Endpoint	Response / behavior
GET /api/v1/health	Service readiness and last successful publication time; no internal secrets.
GET /api/v1/index/latest	Latest published daily release for explicit dataset/series; stale age and status.
GET /api/v1/index/series	Date bounds, daily/weekly/monthly frequency, series ID, vintage; bounded pagination.
GET /api/v1/basket	Versioned route/product/window weights and provenance.
GET /api/v1/coverage	Observed, imputed, unavailable and failed weights by route/source/horizon.
GET /api/v1/releases/{id}	Method versions, lineage summary, manifest hash and revision reason.
GET /api/v1/validation	Independent benchmark metadata and actual comparable-period metrics.
GET /api/v1/exports	Bounded CSV/JSON download with release/version metadata.

Query defaults may select LIVE, but they must never fall back to SYNTHETIC when live data is absent. Return a clear no-data or suppressed response. Use 400 for invalid ranges, 401/403 for protected access failures, 404 for unknown IDs, 429 for client quotas and 503 for an unavailable dependency. Avoid returning a fabricated zero index with HTTP 200.

```
{
  "series_id": "apix-pilot-payable",
  "date": "2026-10-15",
  "value": 107.6992,
  "dataset_kind": "SYNTHETIC",
  "status": "DEMO",
  "observed_weight": 1.0,
  "imputed_weight": 0.0,
  "basket_version": "example-v1",
  "release_id": "example-only-r1"
}
```

This JSON is an illustrative contract, not a real result. Actual responses also include baseline, method version, observation cutoff, units and revision.

Write access and existing authentication

Keep aggregate reads separate from source-policy changes, run requests, selector approvals and publication. Use scoped service credentials for the initial operator CLI; never embed them in React. The existing Convex login can remain for the demo UI, but does not automatically authorize the new API. Before enabling browser-based admin operations, implement and test an explicit trusted token-verification/role-mapping path or an OIDC provider shared by both services. Until then, admin controls call the operator CLI through the authorized operator, not an insecure frontend token.

Provide OpenAPI and examples for NSO/RBI-style consumption. Call the JSON format application-specific; do not claim SDMX compliance without an actual structure definition, mapping and conformance validation.

19 / Dashboard that shows the quality of the number

Reuse the existing page structure

Existing page	Revised behavior
Overview	Daily index, explicit base period, dataset badge, release time, observed/imputed coverage, optional seven-day average.
Sectors	Route x period heatmap; contribution in index points; suppressed cells visibly distinct from zero change.
LeadTime	Five-window prices/ratios with sample and availability counts; descriptive interpretation.
Pipeline	Actual jobs and source health: allowed, paused, blocked, drifted, complete; request timing and quota usage.
Explorer	Raw quote → normalized components → eligibility reason → slot; protected evidence preview.
Validation	Authentic date range, benchmark file provenance, comparable units, period count and unfulfilled requirements.
Methodology	Basket/product definitions, assumptions, formulas, imputation rules and version history.
API documentation	Real response examples, release pinning, stale/no-data behavior, authentication scopes.
Deck	Evidence-backed milestones and a clearly labeled demo fallback.
Drivers	Sourced event context only; hide simulated regression coefficients from live mode.

Four essential user journeys

Analyst: open a date, inspect its coverage, choose a route and lead window, inspect a contributing offer and its payable-price components. The same release ID follows the user through every screen.

Collector operator: see a source enter PARSE_DRIFT, inspect evidence and selector differences, run a fixture replay, approve a new selector version and observe a candidate release revision.

Methodology reviewer: compare alternative channel/lead weights, see impact on the index, and publish a new method version without rewriting old releases.

API consumer: download a pinned release and its manifest, reproduce the aggregate independently, and determine whether a value is live, archived, synthetic, provisional or suppressed.

Presentation rules

Persist the dataset badge on every chart, table and export. Show “No observed data yet” when appropriate. Separate no service, sold out and collection failure. A source-health badge is not a permission badge; both need their own status. Do not show an accuracy percentage unless its denominator, independently labeled evidence and calculation are available.

“Real-time” should describe up-to-date acquisition visibility and frequent releases, with a visible schedule and timestamp. It must not imply continuous universal fare coverage. Keep implementation internals on the operator screens; analyst views should explain what a number means and how trustworthy it is.

20 / The 30-day validation plan

Two independent clocks

Engineering clock: a 36-hour sprint can produce adapters, replay, an index engine, API and demonstration.

Evidence clock: 30 days of authentic historical price observations require either an existing permitted archive or 30 days of collection. A synthetic generator cannot accelerate that requirement.

Start the first permitted minimal collector as soon as policy and evidence storage work. Preserve its raw observations even while the rest of the system is being built. If starting on 30 September, 30 consecutive observation dates end on 29 October; a complete October monthly comparison needs observations through 31 October plus the later benchmark release. Missed dates may extend the usable evidence period.

Back-test protocol

1. Obtain at least 30 distinct observation dates from genuine archived captures or prospective collection. For archives, record original capture times, source rights and completeness; a modern replay timestamp is not a historical quote date. 2. Import the independent DGCA file. Verify publisher, scope, unit, routes, passenger/cabin coverage, taxes, booking versus travel month and revision/vintage. 3. Reconstruct APIx from frozen raw evidence with an explicit baseline and method. Archive replay validates software reproducibility; claim vintage-realistic back-testing only when the inputs and rules available at each historical date are preserved. 4. If DGCA provides INR averages, construct a separate comparable APIx mean-fare statistic; never subtract an index number from an INR price. Compare movements only after consistent rebasing and scope alignment. 5. Publish monthly observations as monthly. Do not repeat/interpolate a DGCA monthly value into 30 daily points and claim 30 independent benchmark observations. 6. Report matched periods, exclusions, coverage and discrepancies. With one monthly pair, show level difference only where meaningful; correlation, trend accuracy and forecast skill are not defensible.

Metrics and limitations

For adequately matched multiple periods, report absolute/percentage fare error, change-direction agreement and correlation with sample size. For INR fares, MAPE =

$\text{mean}(\text{abs}(\text{predicted} - \text{benchmark}) / \text{benchmark}) \times 100$ for nonzero benchmarks. For rebased index comparison, report index-point or percentage-change differences separately. Avoid an arbitrary “PASS” label based only on correlation.

A proposed research threshold is at least 12 overlapping monthly periods for descriptive stability checks and at least 24 training months plus a separately held-out period before trying a simple nowcast. These are project research gates, not guarantees of statistical sufficiency. Compare forecasting against seasonal-naïve baselines using rolling-origin evaluation and no future-data leakage.

Fallback if benchmark remains inaccessible: complete all engineering demonstrations, show the authentic collection coverage and label the DGCA requirement outstanding. A supplementary MoSPI airfare comparison remains separately labeled. No invented benchmark and no derivative series masquerading as DGCA.

21 / Test strategy and acceptance gates

Layer	Required tests	Passing evidence
Policy	Disallow, expired permission, missing robots, redirect to unapproved host, kill switch	No disallowed network request; audit contains precise denial reason.
Scheduling	Duplicate trigger, worker death, stale lease completion, 429 cooldown, midnight rollover	No duplicate accepted observation; lease recovery; correct IST date/lead.
Browser/HTTP	JS readiness, wrong query echo, session expiry, timeout, CAPTCHA	Correct classification; no retry loop or wrong-route quote.
Extraction	CSS drift, valid selector matching wrong number, multiple families, currency decoys	Correct fields or abstention; no silent wrong fare.
Normalization	INR decimals, missing component, fee at checkout, coupon eligibility, baggage upgrade	Exact paise arithmetic; unknown != zero; reference product preserved.
Quality	Genuine 2x price shock, duplicate cards, sold-out, cancellation, empty results	Shock retained when valid; states remain distinct.
Index	All relatives 1; all prices $\times 1.10$; numerical worked example; reordered input	100; 110; 107.6992 within 0.0001; order-independent output.
Missingness	Source outage, 20% vs >20% imputation, absent donor, third missing day	Declared thresholds enforced; no weight renormalization or synthetic fill.
Release/API	Rerun same manifest, changed selector, partial month, missing comparator, mixed dataset kinds	Stable revision; explicit new revision; null/suppressed states; synthetic excluded.
Validation	Independent fixture file, differing units, one monthly pair, future benchmark vintage	Scope guard; no false correlation/accuracy claim; no look-ahead.
UI	Badge across navigation/export, drill-down, blocked-source state, missing release	User sees origin, freshness, coverage and limitation consistently.

Measures to report, not invent

Proposed pilot goals: at least 99% exact total-price extraction on a manually labeled supported-layout fixture set; zero wrong-route/date/currency acceptances on the hard-negative set; 100% retained lineage for published cells; deterministic replay to 0.0001 index points; and publication within 60 minutes of the last configured window. These are targets until measured.

Report extraction precision, coverage and availability separately. A source can be unavailable while the parser remains correct. A parser that abstains on most pages can have high precision but poor usefulness, so include recall/abstention and source-level denominators.

Use local browser fixtures and stored permitted evidence in CI. External sites must not be hammered by every test run. A separately authorized nightly canary checks current source drift within its own rate budget. Keep smoke-test observations out of the statistical sample unless they meet the published sampling design.

22 / Deployment, capacity and cost model

Pilot deployment

Run API, collector worker, scheduler and PostgreSQL as separate processes; use container packaging during implementation if desired. Serve the built React app through a reverse proxy. Keep browser work away from the API process. Start with a 4-vCPU, 8-GB-RAM host and two browser contexts globally as an

unbenchmarked starting envelope, then measure memory and latency.

Use TLS, scoped credentials, a secrets store/environment injection, parameterized database access and allowed-host egress checks. No arbitrary user-provided URL fetching. Authenticate operator actions and redact cookies/tokens from evidence and logs. Keep evidence private and sanitize HTML previews. Do not expose database or browser debugging ports publicly.

Capacity arithmetic

Logical searches/day = directed routes × lead windows × sweeps × sources

Pilot upper bound = $12 \times 5 \times 1 \times 2 = 120$

Expanded upper bound = $50 \times 5 \times 3 \times 11 = 8,250$

These are upper bounds before route/source eligibility, not quote counts. A search can yield many flight offers and many network requests. With a modeled 30-second browser search, the pilot needs 1 browser-hour/day; the expanded plan needs 68.75 browser-hours/day. Two contexts provide only 48 theoretical context-hours/day before downtime, so that expanded case needs more capacity or a more efficient permitted transport. Four contexts need about 17.2 wall-clock hours at ideal utilization; the observation windows may be the tighter constraint.

At one new search every 30 seconds per source, a two-hour window permits at most 240 starts/source, before subrequest limits and downtime. A 50-route × 5-window sweep needs 250, so it does not fit that default. Widen the registered window, reduce the sampling workload with PSD approval, or obtain a higher permitted rate; do not add IPs to evade the quota. Extra workers only solve compute constraints.

At 200 KB compressed evidence per search, 120 searches/day over 30 days is about 720 MB; 8,250 searches/day is about 49.5 GB. Screenshots, retries, database rows and backups are additional. Measure bytes/search and replace these assumptions.

Operational and budget controls

Budget = compute hours × chosen host rate + retained GB-months + egress + backups + any permitted data licenses. No vendor prices are asserted here. Track cost per accepted observation and per published date. Agree retention with source conditions; proposed pilot evidence retention is 90 days where permitted, with published aggregates/manifests retained longer. Run daily database/evidence backups and a restore drill; initial recovery targets are 24-hour data loss maximum and restoration within 4 hours. Source rules may require stricter retention limits.

23 / Migration from the current project

Keep the interface; replace the data boundary

1. Preserve the current repository and user changes. Do not restore the deleted scraper/safar files automatically. Add the new Python package under `services/safar/` so the new implementation is distinguishable from the removed reference collector. 2. Isolate the existing TypeScript generator behind `dataset_kind=SYNTHETIC`. Remove live claims from the simulation path, especially the internally manufactured DGCA benchmark. 3. Build real collection and normalization with fixture playback first. Load the same API contracts into a separate frontend data client, `src/lib/apix-client.ts`. 4. Migrate read pages to release-backed data. Retain Convex authentication temporarily, with the write-access constraint in section 18. Do not maintain two independent authoritative index engines. 5. Keep old demo views available only under an explicit demo mode. On cutover, production routes select published API results or show no data. 6. Backfill only authentic archive observations with preserved original times and dataset provenance. Verify a release replay before enabling publication.

Proposed repository additions

```
services/safar/pyproject.toml          # pinned dependency/CLI project
services/safar/src/safar/
  contracts.py                         # jobs, evidence, quotes, releases
  collect/{planner,jobs,policy,sessions,worker}.py
  collect/transport/{http,browser,replay}.py
  adapters/{base,registry,airline_pilot,ota_pilot}.py
  extract/{selectors,fingerprints,recovery}.py
  normalize/{money,components,products,quality}.py
  index/{basket,cells,weights,missing,compute,release}.py
  validation/{imports,scope,metrics}.py
  storage/{models,evidence,repository}.py
  api/{app,reads,auth}.py
  cli.py
services/safar/tests/{unit,integration,fixtures}/
services/safar/migrations/
config/{sources,baskets,methods}/
data_sources/register.csv
src/lib/apix-client.ts
```

The pilot adapter filenames are placeholders for roles, not claimed source integrations. At source onboarding, assign an exact source-specific module name and registry ID only after permission and technical feasibility checks. Every later adapter must satisfy the same contract and fixture suite.

Design ownership: one owner for collection, one for extraction/normalization, one for methodology/data, one for API/storage, one for frontend and one for testing/integration. This is a six-person planning assumption; combine roles if team size differs.

24 / Implementation tasks: foundation and collection

All paths below are relative to `services/safar/` unless prefixed `src/`, `config/` or `data_sources/`. Tasks are planned work; the commands are acceptance commands to create and run during implementation, not commands claimed to pass now. For each task: write the named failing tests, confirm failure, implement the defined interface, rerun to green, review the diff and commit the task.

T1 — Typed records, policy and evidence store

Create: `src/safar/contracts.py`, `storage/models.py`, `storage/evidence.py`, `collect/policy.py`, initial migrations, `config/sources/`, `tests/unit/test_policy.py`, `tests/unit/test_money.py`, `tests/integration/test_evidence.py`. Add the project file and pinned environment here.

Interfaces: `PolicyGate.evaluate(job, request_url) -> PolicyDecision`; `EvidenceStore.put(content, metadata) -> EvidenceRef`; `parse_money(text, currency) -> int | None`. Persist deny reasons, content hashes and redaction status.

Checks: denied policy makes zero transport calls; unknown amount returns null; INR 6,500.50 becomes 650050 paise; identical redacted content produces the same hash; stored evidence cannot be mutated through the normal API. Run `python -m pytest tests/unit/test_policy.py tests/unit/test_money.py tests/integration/test_evidence.py -q`.

T2 — Planner, durable jobs and replay transport

Create: `collect/planner.py`, `collect/jobs.py`, `collect/worker.py`, `collect/transport/replay.py`, `tests/integration/test_jobs.py`, `tests/unit/test_planner.py`.

Interfaces: `Planner.plan(basket, observation_window) -> list[SearchJob]`; `JobRepository.claim(worker_id, now) -> LeasedJob | None`; `complete(job_id, lease_token, result) -> bool`. Generate all five horizons and include the complete query context in idempotency identity.

Checks: 12 directions × 5 horizons × 1 sweep × 2 eligible sources = 120 jobs; repeated planning creates no duplicates; a crashed lease can be reclaimed; an old token cannot finish; midnight crossing expires the old job. Run `python -m pytest tests/unit/test_planner.py tests/integration/test_jobs.py -q`.

T3 — One airline adapter, then one OTA

Create: `collect/transport/http.py`, `collect/transport/browser.py`, `collect/sessions.py`, `adapters/base.py`, `adapters/registry.py`, selected source modules and corresponding `tests/integration/test_adapters.py` fixtures.

Interfaces: `Transport.fetch(job, session) -> FetchResult`; `SourceAdapter.extract(fetch_result, job) -> ExtractionBatch`. Consume T1 policy/evidence and T2 jobs. Acquire only after policy acceptance; verify search echo and price-product identity.

Checks: JS result readiness, wrong departure date, expired session, 429, CAPTCHA, fee-incomplete OTA and complete direct quote. Run `python -m pytest tests/integration/test_adapters.py -q`. Then conduct one permitted live canary per onboarded source and record the evidence ID. No canary success means no claimed live integration.

25 / Implementation tasks: extraction and statistics

T4 — Normalization, availability and deduplication

Create: `normalize/{money,components,products,quality}.py`, normalization migrations, `tests/unit/test_normalization.py`, `tests/unit/test_quality.py`.

Interfaces: `Normalizer.normalize(raw_quote, method) -> NormalizedQuote`; `QualityGate.evaluate(quote, history) -> QualityDecision`. Consume T3 raw evidence. Preserve null components, eligibility reasons and channel-specific offers.

Checks: section 12's example equals 510000 paise; unknown mandatory fee is ineligible; bundled baggage is never subtracted; a duplicate card collapses while another OTA's offer remains; valid 2x shock is retained; sold-out has no price. Run `python -m pytest tests/unit/test_normalization.py tests/unit/test_quality.py -q`.

T5 — Selector drift and reviewed recovery

Create: `extract/{selectors,fingerprints,recovery}.py`, selector-version migrations, `tests/unit/test_recovery.py`, labeled tests/fixtures/drift/, recovery report CLI.

Interfaces: `Recovery.propose(evidence, approved_fingerprint, job) -> RecoveryProposal`; `SelectorRegistry.approve(proposal_id, reviewer, fixture_report) -> SelectorVersion`. Store candidates separately from eligible quotes.

Checks: class rename recovers a candidate; ambiguity gap below 0.15 abstains; wrong currency/itinerary fails even at high similarity; no automatic promotion; replays retain capture time. Run `python -m pytest tests/unit/test_recovery.py -q`. Report precision/abstention on a held-out fixture set before promotion.

T6 — Basket, cell estimator and missingness

Create: `index/{basket,cells,weights,missing,compute}.py`, baseline/basket migrations, `config/baskets/`, `config/methods/`, `tests/unit/test_index.py`, `tests/unit/test_missing.py`.

Interfaces: `BasketLoader.load(file) -> BasketVersion`; `CellBuilder.build(quotes, basket, date) -> list[CellValue]`; `compute(cells, baseline, weights) -> IndexResult`. Use exact section 15 formulas and section 16 gates; return suppression reasons as data.

Checks: invalid weight sum rejected; baseline 100; common 10% rise gives 110; worked fixture 107.6992; no input-order dependence; outage cannot alter fixed weights; third consecutive imputation suppresses; no future donor. Run `python -m pytest tests/unit/test_index.py tests/unit/test_missing.py -q`.

T7 — Releases and independent validation

Create: `index/release.py`, `validation/{imports,scope,metrics}.py`, `data_sources/register.csv`, `tests/integration/test_releases.py`, `tests/unit/test_validation.py`.

Interfaces: `ReleaseBuilder.build(manifest) -> CandidateRelease`; `Publisher.publish(candidate_id, reviewer) -> Release`; `BenchmarkImporter.load(file, metadata) -> BenchmarkImport`. Reject mixed synthetic/live manifests and incompatible comparison units.

Checks: same manifest is deterministic; change creates an explicit revision; partial month is provisional; one monthly pair has null correlation; benchmark values cannot be generated from APIx outputs. Run `python -m pytest tests/integration/test_releases.py tests/unit/test_validation.py -q`.

26 / Implementation tasks: serving and proof

T8 — API and dashboard migration

Create: `api/{app,reads,auth}.py`, `tests/integration/test_api.py`, `root src/lib/apix-client.ts`. **Modify:** existing Overview, Sectors, LeadTime, Explorer, Pipeline, Validation, Methodology and ApiDocs pages to use pinned release responses.

Interfaces: endpoints in section 18; typed frontend client `getSeries(params)`, `getRelease(id)` and `getCoverage(params)`. Keep admin writes in the scoped operator CLI until a trusted user-token integration is implemented and tested.

Checks: unauthenticated protected access fails; public aggregates cannot mutate policy; missing LIVE data never returns SYNTHETIC data; date pagination is stable; every chart/export shows dataset and release metadata; one index point drills to its cell and evidence. Run `python -m pytest tests/integration/test_api.py -q`, then the existing frontend typecheck/build command established from `package.json` and a real browser walkthrough.

T9 — Scheduler, runbooks and recovery

Create: `cli.py`, deployment configuration, `docs/runbook.md`, `docs/adapters-handbook.md`, `tests/integration/test_pipeline_e2e.py`. Document the schedule, secrets, backup, restore, source suspension, publication and rollback procedures.

Interfaces: proposed CLI commands `safar plan`, `safar worker`, `safar replay`, `safar build-release`, `safar publish` and `safar import-benchmark`. Every command prints IDs and machine-readable status; none labels replay as new collection.

Checks: full fixture pipeline produces one reproducible release; terminate/restart a worker without duplicate accepted quotes; restore database plus evidence to a new environment and reproduce the manifest hash. Run `python -m pytest tests/integration/test_pipeline_e2e.py -q`, then perform and retain the restore drill log.

T10 — Broader source coverage and 30-day evidence

Add the remaining named sources one at a time through T3's onboarding contract. Maintain a per-source matrix: requested, permission pending, fixture ready, live verified, paused. For each new adapter, run its fixtures and permitted canary; add it to a new sampling/basket version only after baseline compatibility is established.

Collect authentic observations across at least 30 dates; import and align the independent benchmark; publish the validation report with all limitations. This task is calendar- and data-dependent and cannot be collapsed into the hackathon sprint.

Final review focus

Explicitly review these high-risk conditions across tasks: valid selector returning a wrong amount (T3/T5); source disappearance changing the basket (T6); fee/baggage mismatch across channels (T4); UTC/IST date rollover (T2); synthetic values leaking into live publication or benchmark validation (T7/T8). A passing unit-test count without these conditions does not establish readiness.

Before submission, run the full service test suite, frontend build and end-to-end walkthrough; inspect actual outputs and attach the run report. Do not reuse this document's proposed targets as measured results.

27 / Team schedule, risks and judging demonstration

Delivery sequence

Time	Team focus	Demonstrable exit
Hours 0–6	Data owner verifies inputs; collection/storage owners implement T1–T2; frontend owner isolates demo labels	Source register, policy denial, durable replay jobs; no fabricated live claims.
Hours 6–16	Collection + extraction owners T3–T4; methodology owner prepares T6 fixtures; API owner contracts	One permitted source observation if feasible; normalized fixture trace.
Hours 16–26	Second source; estimator/releases; API/UI integration	Candidate daily release; no-data/coverage states; drill-down.
Hours 26–36	T5 recovery demo, T8–T9 integration, test/report owner rehearses	Reproducible demo package and transparent remaining gaps.
Days 2–7	Stabilize canaries, fix source drift, freeze seven-day baseline	Evidence-backed pilot with monitoring and initial releases.
Days 8–30+	Expand only permitted coverage; accumulate authentic history; obtain DGCA file	Thirty-date manifest and appropriately scoped independent comparison.
After evidence gate	Statistical review, sampling expansion, official configuration, research features	Credible pilot proposal for PSD review, not a self-declared official CPI replacement.

This schedule assumes six contributors and some permitted source access. If permissions or the independent benchmark are unavailable, complete engineering evidence and report the affected PS deliverables as outstanding. The recovery feature may be demonstrated on replay before it is ready for live use.

Principal risks and responses

Source access denied: suspend; pursue approved feeds/permissions; expose lost coverage. **DOM change:** quarantine, propose repair, replay, review. **Mandatory fee inaccessible:** retain headline-only quote, exclude payable-price claim. **Benchmark missing:** escalate the data request; never synthesize the reference. **Scope growth:** defer chatbot, GEKS and nowcast. **Compute insufficient:** measure per-search cost and narrow or resource the approved sampling plan; never bypass quotas.

Six-minute demo

0:00–0:45 — State PS and show the authentic date range and data badge. 0:45–1:45 — Follow a permitted quote from saved evidence through fee/product normalization. 1:45–2:45 — Replay a local DOM change and a decoy-price case; show repair candidate and abstention. 2:45–3:30 — Trigger a policy denial and source outage; show missing-weight behavior. 3:30–4:30 — Explain the worked index and route contribution. 4:30–5:15 — Fetch a pinned release through the API and its lineage. 5:15–6:00 — Show independent validation status and the next evidence gate.

Recommended pitch: “We built an airfare-specific collection and statistical pipeline that can explain every accepted price, detect when extraction becomes unreliable, and reproduce every published index.” Say it only when the corresponding evidence exists.

28 / References: local inputs and Scrapling

All external references were checked on 30 September 2026. Repository links below pin the studied revision so future upstream changes do not silently change this document's basis. Source findings are distinguished from proposed APIx design decisions throughout.

[L1] User-supplied problem statement. [ps.md](#), PS 26056. Authority for the requested deliverables and named sources. Background market statistics were not independently verified here.

[L2] Previous architecture plan. [APIx_SIH2026_Architecture_Plan_v2.pdf](#), sections 1–10. Reviewed for architecture, normalization, index design and back-test strategy.

[L3] Repository inspection. Commit 683bbcc plus working-tree state on review date. Files inspected include `src/convex/lib/{engine,index,basket,adapters}.ts`, `src/convex/pipeline.ts`, `src/convex/schema.ts`, `scraper/README.md`, `scraper/requirements.txt`, `scraper/tests/test_pipeline.py`, README and frontend page inventory. Existing deletions were left intact.

[S1] Scrapling repository and feature overview. `Fetcher/session/parser/crawler` separation is the conceptual reference; performance and anti-bot claims are upstream claims, not measured APIx results.

[Repository at reviewed revision](#)

[S2] Scrapling parser implementation. Adaptive relocation and similarity-based selection; informs the design problem, not the proposed APIx thresholds. [parser.py](#)

[S3] Scrapling descriptor storage. Storage abstraction and SQLite implementation for saved element properties. [core/storage.py](#)

[S4] Scrapling scheduler. Request deduplication, pending/in-flight tracking and snapshot/restore behavior. APIx instead proposes durable database leases with airfare query identity. [spiders/scheduler.py](#)

[S5] Scrapling session manager. Named session registration and lifecycle management. [spiders/session.py](#)

[S6] Scrapling license at reviewed revision. BSD 3-Clause; preserve required notices if code is subsequently reused. The present plan calls for an independent implementation and includes no upstream source code. [LICENSE](#)

29 / References: statistics and engineering

[S7] MoSPI, First press release of CPI on base 2024=100, 12 February 2026. Pages 1 and 5–6 establish the new base, HCES weight reference, classification change and digital/administrative coverage. This corrects the current-context framing; it does not establish the correct airfare weight for this project.

[Official CPI release and technical note](#)

[S8] eSankhyiki CPI catalogue. Discovery portal for official indices and metadata; individual item tables must still be downloaded and verified. [MoSPI CPI catalogue](#)

[S9] RFC 9309 — Robots Exclusion Protocol. Basis for distinguishing crawler rules from authorization. The project's fail-closed treatment of unavailable policy evidence is an additional conservative design rule.

[RFC 9309](#)

[S10] Playwright Python network documentation. Documents network events, request/response access and routing. It does not authorize use against a source. [Network documentation](#)

[S11] PostgreSQL SELECT documentation. Row locking and SKIP LOCKED behavior support the proposed worker-claim pattern. Worker leases and idempotency remain application responsibilities. [PostgreSQL SELECT](#)

[SELECT](#)

[S12] DGCA domestic traffic portal — attempted discovery. The portal could not be retrieved through the research browser. No exact public monthly average-fare series or OD weight file was verified; the plan treats both as dependencies rather than invented inputs. [DGCA traffic data entry point](#)

[S13] Consumer Price Index Manual: Concepts and Methods, 2020. General reference for elementary aggregation, expenditure weighting, missing prices, quality changes and linking. The equations, thresholds and sampling choices in this document are a proposed experimental design, not an assertion of MoSPI approval. [IMF publication and chapter catalogue](#)

Final submission checklist

- Identify which sources have genuine permission and live verification.
- Attach the exact PSD basket/weight configuration or label the experimental substitute.
- State the true authentic history length and benchmark availability.
- Export a release with its method, coverage, input manifest and lineage.
- Include actual test results and the recovery/abstention evaluation.
- Label synthetic demonstrations and keep them outside live statistics.
- Present source adaptation and statistical auditability as measurable capabilities.
- Defer official CPI integration claims until the relevant methodology is agreed with PSD.